

RFC-0001 — Components Vision and Terminology

Status: Draft

Category: Architecture

Version: 0.1

1. Purpose

This document defines the core terminology and architectural vision of the Components Platform.

Every specification, API, package, Board, AI integration, and implementation **MUST** follow the terminology defined in this document.

This RFC intentionally defines concepts rather than implementation details.

2. Vision

Components is a headless platform for electronic design, learning, simulation, validation, automation, and AI-assisted construction.

Components is not a graphical application.

Components is not tied to:

- a particular operating system
- a particular user interface
- a particular programming language
- a particular AI provider
- a particular renderer
- a particular storage backend

Components is intended to become the common foundation shared by:

- humans
 - AI agents
 - command-line tools
 - graphical editors
 - educational tools
 - automated generators
 - future hardware design environments
-

3. Philosophy

Components does not describe how to draw a circuit.

Components describes what the circuit is.

Presentation belongs to Boards.

Meaning belongs to Components.

4. Core Concepts

The Components ecosystem consists of three user-facing concepts.

Component

A Component describes the electrical and logical structure of a design.

It defines:

- components
- ports
- connections
- buses
- interfaces
- hierarchy

A Component never describes visual layout.

Board

A Board describes how a Component is presented.

A Board may define:

- placement
- routing
- grouping
- visibility
- labels
- themes
- learning views

- engineering views
- breadboard views

A Board never changes electrical connectivity.

Multiple Boards may represent the same Component.

Resources

Resources describe everything a project depends upon.

Examples include:

- libraries
- modules
- themes
- symbols
- datasheets
- lessons
- documentation
- images
- tests

Resources may extend presentation and learning.

Resources may not redefine electrical truth.

5. Components Platform

Components Platform is the authoritative execution engine.

It owns:

- package loading
- dependency resolution
- validation
- simulation
- export
- history
- transactions
- revisions

Every client communicates with the Platform.

No client owns circuit semantics.

6. Board

A Board is a client.

A Board may be implemented as:

- Web application
- Native desktop application
- Tablet application
- Mobile application
- VS Code extension
- Jupyter extension
- Terminal interface
- AR/VR interface

Boards are replaceable.

Boards do not define Components.

7. Package

A Package is the portable document exchanged between clients and the platform.

A Package contains:

- Component
- one or more Boards
- Resources
- metadata

Generated files are not considered source.

8. Component Language

Component Language is the declarative language used to describe circuit structure.

It is:

- structural

- functional
- deterministic
- AI-friendly
- voice-friendly
- student-friendly

Component Language is the canonical source of circuit meaning.

9. Board Language

Board Language describes visual presentation.

It controls:

- layout
- routing
- grouping
- themes
- visibility
- interaction hints

Board Language never changes circuit meaning.

10. Operation

Every modification to a project is represented as an Operation.

Examples:

- connect
- disconnect
- place
- route
- rename
- group
- import
- export

Operations are:

- structured
- validated
- attributable
- reversible

Operations form the basis for:

- undo
 - redo
 - replay
 - history
 - collaboration
 - AI interaction
-

11. Session

A Session represents an active participant interacting with Components.

A Session may represent:

- a human
- an AI
- a script
- a CLI
- a Board
- an automated service

Every Operation belongs to exactly one Session.

12. Provider

A Provider extends Components without changing its architecture.

Examples include:

- AI Provider
- Storage Provider
- Export Provider
- Compiler Provider
- Simulation Provider

Providers are replaceable.

13. AI Router

An AI Router translates natural interaction into Components Operations.

It may accept:

- text
- voice
- images
- multimodal input

It may use:

- local models
- cloud models
- school-hosted models

The Platform never depends on a particular AI provider.

14. Headless Principle

Components must remain fully functional without any graphical interface.

Every feature available through a Board should also be accessible through platform interfaces.

15. Canonical Truth

Only the following are considered primary sources:

- Component
- Board
- Resources
- Package metadata

Everything else is derived.

Examples of derived artifacts:

- rendered graphics
- simulation cache
- generated Verilog
- generated SystemVerilog
- generated documentation

Derived artifacts may be regenerated at any time.

16. Replaceability

The following should be replaceable:

- Boards
- Renderers
- AI providers
- Programming languages
- Storage systems
- Export implementations

The following should remain stable:

- Component Language
 - Package format
 - Operation model
 - Architectural principles
-

17. Principles

1. Components is a platform, not an application.
2. Components owns the truth.
3. Boards own the experience.
4. Component defines meaning.
5. Board defines presentation.
6. Resources define dependencies.
7. Every client communicates through the same Operations.
8. AI is a client of the platform.
9. The platform is useful with or without AI.
10. Generated artifacts are never source.
11. Physical and logical identities must remain linked.
12. Architecture comes before implementation.